

Floating Point

Introduction to Computer Systems

Instructors:

Yili Gong

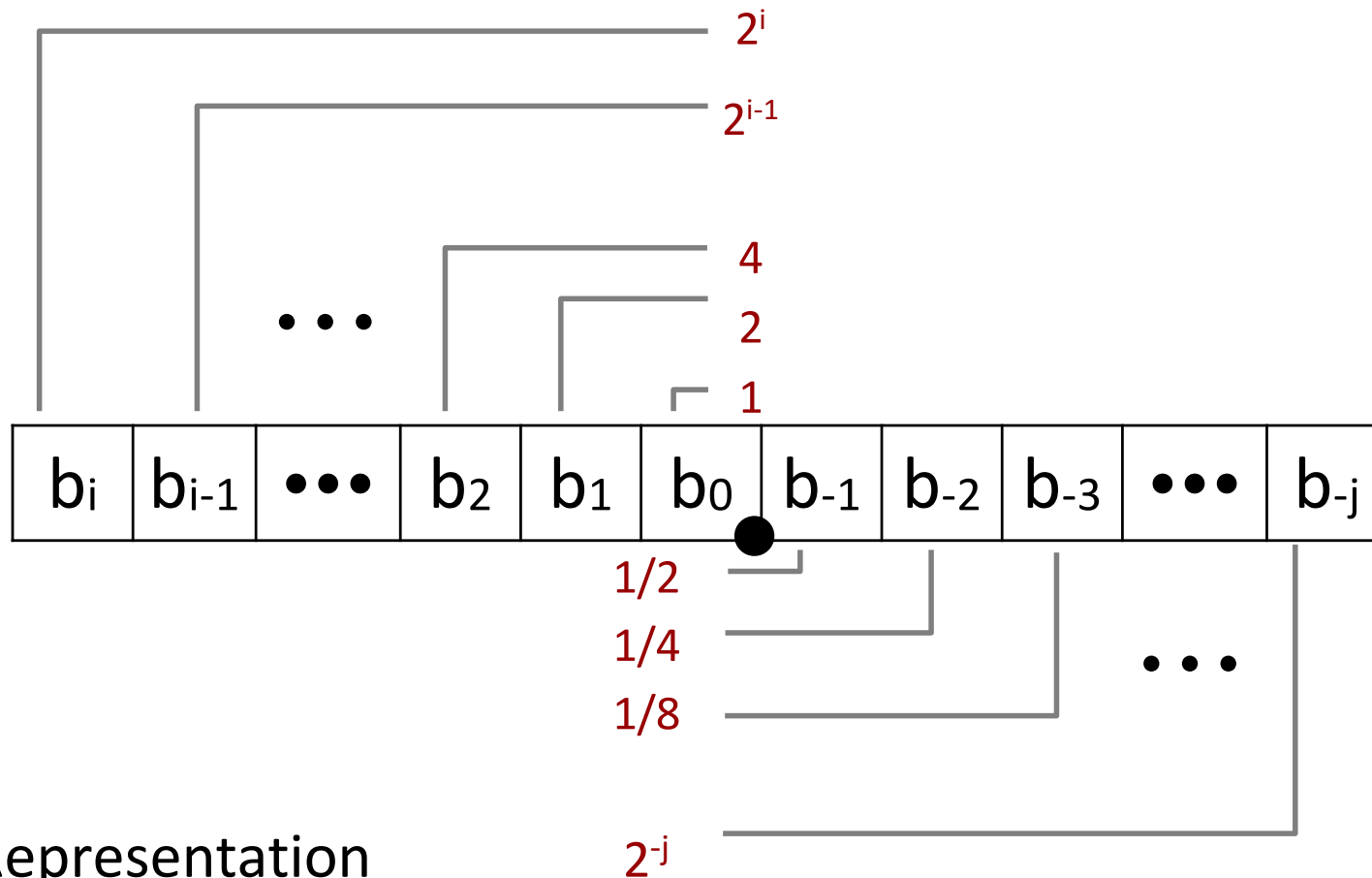
Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

Fractional binary numbers

■ What is 1011.101_2 ?

Fractional Binary Numbers



■ Representation

- Bits to right of “binary point” represent fractional powers of 2
- Represents rational number:

$$\sum_{k=-j}^i b_k \times 2^k$$

Fractional Binary Numbers: Examples

■ Value Representation

$5 \frac{3}{4}$	101.11_2
$2 \frac{7}{8}$	10.111_2
$1 \frac{7}{16}$	1.0111_2

■ Observations

- Divide by 2 by shifting right (unsigned)
- Multiply by 2 by shifting left
- Numbers of form $0.111111..._2$ are just below 1.0
 - $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^i} + \dots \rightarrow 1.0$
 - Use notation $1.0 - \epsilon$

Representable Numbers

■ Limitation #1

- Can only exactly represent numbers of the form $x/2^k$
 - Other rational numbers have repeating bit representations

■ Value	Representation
■ 1/3	0.0101010101 [01] ... ₂
■ 1/5	0.001100110011 [0011] ... ₂
■ 1/10	0.0001100110011 [0011] ... ₂

■ Limitation #2

- Just one setting of binary point within the w bits
 - Limited range of numbers (very small values? very large?)

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

IEEE Floating Point

■ IEEE Standard 754

- Established in 1985 as uniform standard for floating point arithmetic
 - Before that, many idiosyncratic formats
- Supported by all major CPUs

■ Driven by numerical concerns

- Nice standards for rounding, overflow, underflow
- Hard to make fast in hardware
 - Numerical analysts predominated over hardware designers in defining standard

Floating Point Representation

■ Numerical Form:

$$(-1)^s M 2^E$$

- Sign bit s determines whether number is negative or positive
- Significand M normally a fractional value in range $[1.0, 2.0)$.
- Exponent E weights value by power of two

■ Encoding

- MSB s is sign bit s
- exp field encodes E (but is not equal to E)
- frac field encodes M (but is not equal to M)



Precision options

- Single precision: 32 bits



- Double precision: 64 bits



- Extended precision: 80 bits (Intel only)



“Normalized” Values

$$v = (-1)^s M 2^E$$

- When: $\text{exp} \neq 000\dots 0$ and $\text{exp} \neq 111\dots 1$
- Exponent coded as a biased value: $E = \text{Exp} - \text{Bias}$
 - Exp: unsigned value of exp field
 - Bias = $2^{k-1} - 1$, where k is number of exponent bits
 - Single precision: 127 (Exp: 1...254, E: -126...127)
 - Double precision: 1023 (Exp: 1...2046, E: -1022...1023)
- Significand coded with implied leading 1: $M = 1.\text{xxx}\dots\text{x}_2$
 - xxx...x: bits of frac field
 - Minimum when $\text{frac} = 000\dots 0$ ($M = 1.0$)
 - Maximum when $\text{frac} = 111\dots 1$ ($M = 2.0 - \epsilon$)
 - Get extra leading bit for “free”

Normalized Encoding Example

$$v = (-1)^s M 2^E$$
$$E = \text{Exp} - \text{Bias}$$

■ Value: float $F = 15213.0$;

$$15213_{10} = 11101101101101_2$$
$$= 1.1101101101101_2 \times 2^{13}$$

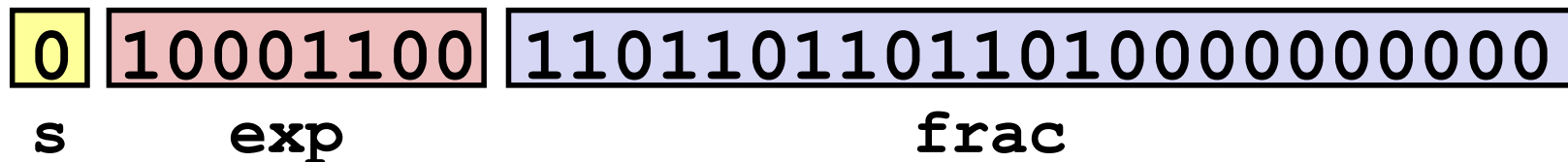
■ Significand

$$M = 1.\underline{1101101101101}_2$$
$$\text{frac} = \underline{110110110110100000000000}_2$$

■ Exponent

$$E = 13$$
$$\text{Bias} = 127$$
$$\text{Exp} = 140 = 10001100_2$$

■ Result:



Denormalized Values

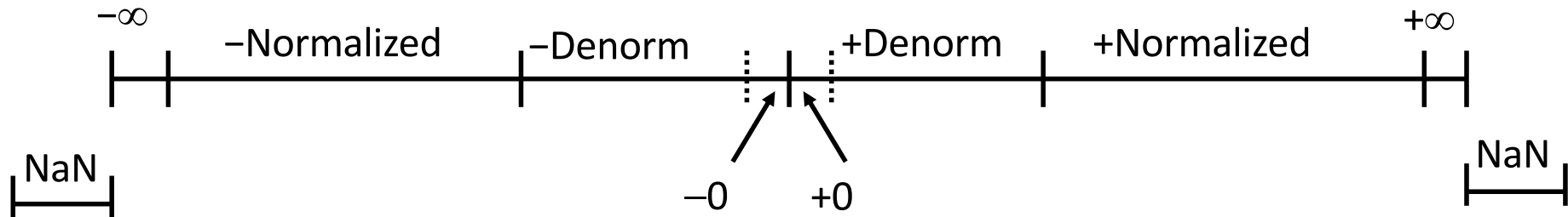
$$v = (-1)^s M 2^E$$
$$E = 1 - \text{Bias}$$

- Condition: $\text{exp} = 000\dots 0$
- Exponent value: $E = 1 - \text{Bias}$ (instead of $E = 0 - \text{Bias}$)
- Significand coded with implied leading 0: $M = 0.\text{xxx}\dots\text{x}_2$
 - **xxx...x**: bits of **frac**
- Cases
 - **exp** = 000...0, **frac** = 000...0
 - Represents zero value
 - Note distinct values: +0 and -0 (why?)
 - **exp** = 000...0, **frac** $\neq$ 000...0
 - Numbers closest to 0.0
 - Equispaced

Special Values

- Condition: **exp** = 111...1
- Case: **exp** = 111...1, **frac** = 000...0
 - Represents value ∞ (infinity)
 - Operation that overflows
 - Both positive and negative
 - E.g., $1.0/0.0 = -1.0/-0.0 = +\infty$, $1.0/-0.0 = -\infty$
- Case: **exp** = 111...1, **frac** $\neq$ 000...0
 - Not-a-Number (NaN)
 - Represents case when no numeric value can be determined
 - E.g., $\text{sqrt}(-1)$, $\infty - \infty$, $\infty \times 0$

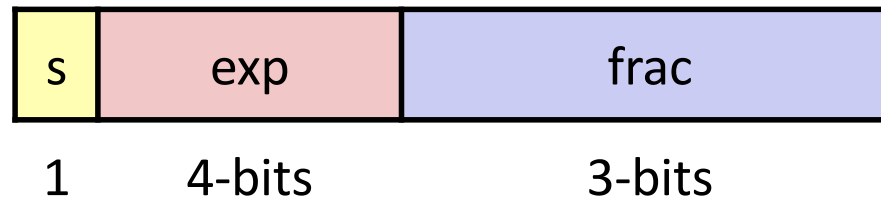
Visualization: Floating Point Encodings



Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

Tiny Floating Point Example



■ 8-bit Floating Point Representation

- the sign bit is in the most significant bit
- the next four bits are the exponent, with a bias of 7
- the last three bits are the **frac**

■ Same general form as IEEE Format

- normalized, denormalized
- representation of 0, NaN, infinity

Dynamic Range (Positive Only)

$$v = (-1)^s M 2^E$$

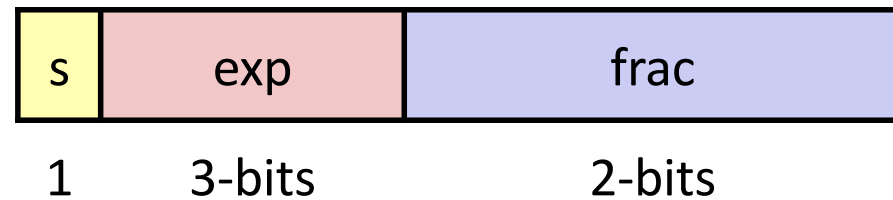
n: $E = \text{Exp} - \text{Bias}$
d: $E = 1 - \text{Bias}$

	s	exp	frac	E	Value	
Denormalized numbers	0	0000	000	-6	0	
	0	0000	001	-6	$1/8 * 1/64 = 1/512$	closest to zero
	0	0000	010	-6	$2/8 * 1/64 = 2/512$	
	...					
	0	0000	110	-6	$6/8 * 1/64 = 6/512$	
	0	0000	111	-6	$7/8 * 1/64 = 7/512$	largest denorm
	0	0001	000	-6	$8/8 * 1/64 = 8/512$	smallest norm
Normalized numbers	0	0001	001	-6	$9/8 * 1/64 = 9/512$	
	...					
	0	0110	110	-1	$14/8 * 1/2 = 14/16$	
	0	0110	111	-1	$15/8 * 1/2 = 15/16$	closest to 1 below
	0	0111	000	0	$8/8 * 1 = 1$	
	0	0111	001	0	$9/8 * 1 = 9/8$	closest to 1 above
	0	0111	010	0	$10/8 * 1 = 10/8$	
	...					
	0	1110	110	7	$14/8 * 128 = 224$	
	0	1110	111	7	$15/8 * 128 = 240$	largest norm
	0	1111	000	n/a	inf	

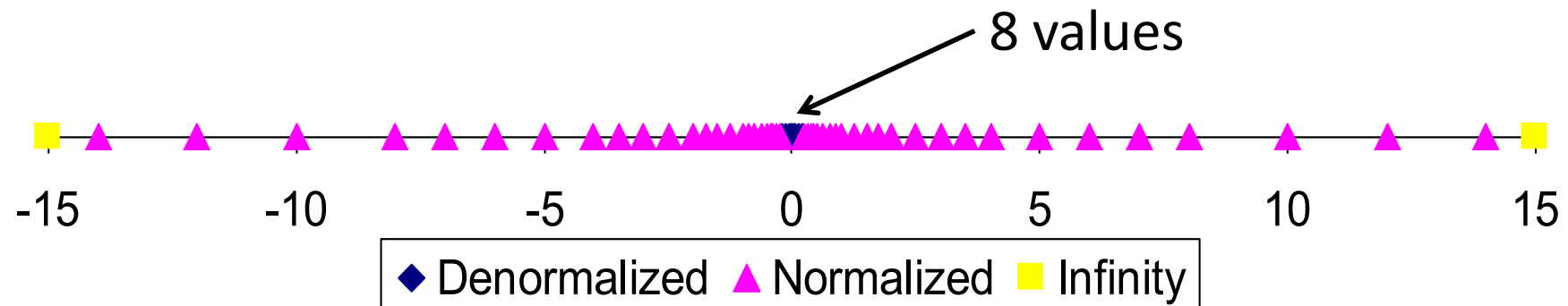
Distribution of Values

■ 6-bit IEEE-like format

- $e = 3$ exponent bits
- $f = 2$ fraction bits
- Bias is $2^{3-1}-1 = 3$



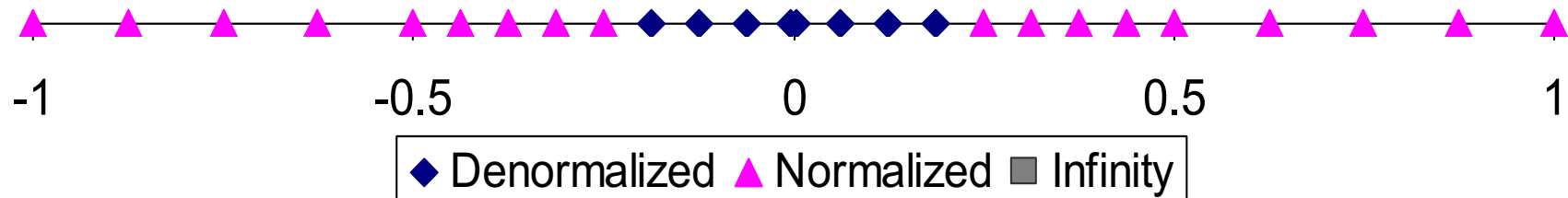
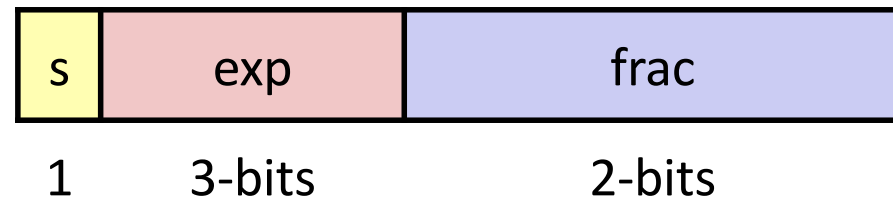
■ Notice how the distribution gets denser toward zero.



Distribution of Values (close-up view)

■ 6-bit IEEE-like format

- $e = 3$ exponent bits
- $f = 2$ fraction bits
- Bias is 3



Special Properties of the IEEE Encoding

- FP Zero Same as Integer Zero
 - All bits = 0
- Can (Almost) Use Unsigned Integer Comparison
 - Must first compare sign bits
 - Must consider $-0 = 0$
 - NaNs problematic
 - Will be greater than any other values
 - What should comparison yield?
 - Otherwise OK
 - Denorm vs. normalized
 - Normalized vs. infinity

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

Floating Point Operations: Basic Idea

■ $x +_f y = \text{Round}(x + y)$

■ $x \times_f y = \text{Round}(x \times y)$

■ Basic idea

- First **compute exact result**
- Make it fit into desired precision
 - Possibly overflow if exponent too large
 - Possibly **round to fit into frac**

Rounding

■ Rounding Modes (illustrate with \$ rounding)

■		\$1.40	\$1.60	\$1.50	\$2.50	−\$1.50
■	Towards zero	\$1	\$1	\$1	\$2	−\$1
■	Round down ($-\infty$)	\$1	\$1	\$1	\$2	−\$2
■	Round up ($+\infty$)	\$2	\$2	\$2	\$3	−\$1
■	Nearest Even (default)	\$1	\$2	\$2	\$2	−\$2

Closer Look at Round-To-Even

■ Default Rounding Mode

- Hard to get any other kind without dropping into assembly
- All others are statistically biased
 - Sum of set of positive numbers will consistently be over- or under-estimated

■ Applying to Other Decimal Places / Bit Positions

- When exactly halfway between two possible values
 - Round so that least significant digit is even
- E.g., round to nearest hundredth

7.8949999	7.89	(Less than half way)
7.8950001	7.90	(Greater than half way)
7.8950000	7.90	(Half way—round up)
7.8850000	7.88	(Half way—round down)

Rounding Binary Numbers

■ Binary Fractional Numbers

- “Even” when least significant bit is 0
- “Half way” when bits to right of rounding position = $100..._2$

■ Examples

- Round to nearest $1/4$ (2 bits right of binary point)

Value	Binary	Rounded	Action	Rounded Value
$2 \frac{3}{32}$	$10.00\textcolor{red}{011}_2$	10.00_2	($<1/2$ —down)	2
$2 \frac{3}{16}$	$10.00\textcolor{red}{110}_2$	10.01_2	($>1/2$ —up)	$2 \frac{1}{4}$
$2 \frac{7}{8}$	$10.11\textcolor{red}{100}_2$	11.00_2	($1/2$ —up)	3
$2 \frac{5}{8}$	$10.10\textcolor{red}{100}_2$	10.10_2	($1/2$ —down)	$2 \frac{1}{2}$

FP Multiplication

- $(-1)^{s1} M1 2^{E1} \times (-1)^{s2} M2 2^{E2}$
- Exact Result: $(-1)^s M 2^E$
 - Sign s: $s1 \wedge s2$
 - Significand M: $M1 \times M2$
 - Exponent E: $E1 + E2$
- Fixing
 - If $M \geq 2$, shift M right, increment E
 - If E out of range, overflow
 - Round M to fit **frac** precision
- Implementation
 - Biggest chore is multiplying significands

Floating Point Addition

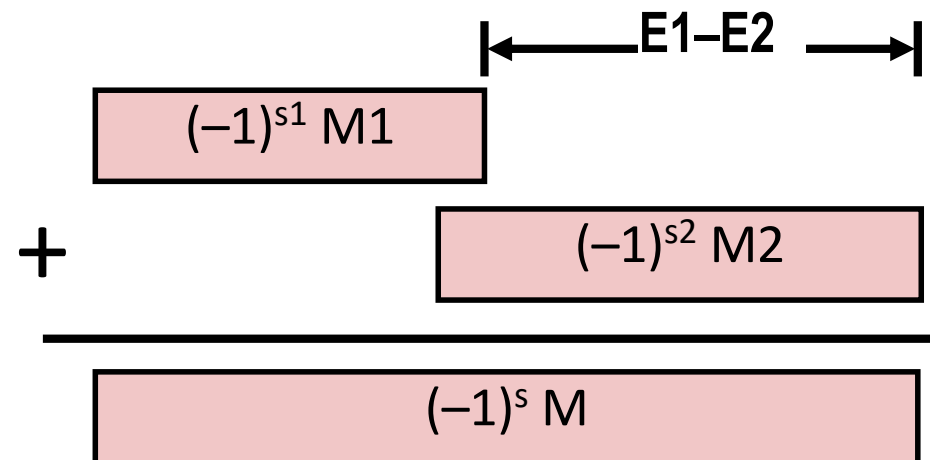
■ $(-1)^{s1} M1 2^{E1} + (-1)^{s2} M2 2^{E2}$

- Assume $E1 > E2$

■ Exact Result: $(-1)^s M 2^E$

- Sign s , significand M :
 - Result of signed align & add
- Exponent E : $E1$

Get binary points lined up



■ Fixing

- If $M \geq 2$, shift M right, increment E
- if $M < 1$, shift M left k positions, decrement E by k
- Overflow if E out of range
- Round M to fit **frac** precision

Mathematical Properties of FP Add

■ Compare to those of Abelian Group

- Closed under addition? **Yes**
 - But may generate infinity or NaN
- Commutative? **Yes**
- Associative? **No**
 - Overflow and inexactness of rounding
 - $(3.14+1e10)-1e10 = 0$, $3.14+(1e10-1e10) = 3.14$
- 0 is additive identity? **Yes**
- Every element has additive inverse? **Almost**
 - Yes, except for infinities & NaNs

■ Monotonicity

- $a \geq b \Rightarrow a+c \geq b+c$ **Almost**
 - Except for infinities & NaNs

Mathematical Properties of FP Mult

■ Compare to Commutative Ring

- Closed under multiplication? Yes
 - But may generate infinity or NaN
- Multiplication Commutative? Yes
- Multiplication is Associative? No
 - Possibility of overflow, inexactness of rounding
 - Ex: $(1e20 * 1e20) * 1e-20 = \text{inf}$, $1e20 * (1e20 * 1e-20) = 1e20$
- 1 is multiplicative identity? Yes
- Multiplication distributes over addition? No
 - Possibility of overflow, inexactness of rounding
 - $1e20 * (1e20 - 1e20) = 0.0$, $1e20 * 1e20 - 1e20 * 1e20 = \text{NaN}$

■ Monotonicity

- $a \geq b \ \& \ c \geq 0 \Rightarrow a * c \geq b * c$ Almost
 - Except for infinities & NaNs

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

Floating Point in C

■ C Guarantees Two Levels

- **float** single precision
- **double** double precision

■ Conversions/Casting

- Casting between **int**, **float**, and **double** changes bit representation
- **double/float** → **int**
 - Truncates fractional part
 - Like rounding toward zero
 - Not defined when out of range or NaN: Generally sets to TMin
- **int** → **double**
 - Exact conversion, as long as **int** has ≤ 53 bit word size
- **int** → **float**
 - Will round according to rounding mode

Floating Point Puzzles

■ For each of the following C expressions, either:

- Argue that it is true for all argument values
- Explain why not true

```
int x = ...;  
float f = ...;  
double d = ...;
```

Assume neither
d nor **f** is NaN

1. `x == (int)(float) x`

2. `x == (int)(double) x`

3. `f == (float)(double) f`

4. `d == (double)(float) d`

5. `f == -(-f);`

6. `2/3 == 2/3.0`

7. `d < 0.0` $\Rightarrow$ `((d*2) < 0.0)`

8. `d > f` $\Rightarrow$ `-f > -d`

9. `d * d >= 0.0`

10. `(d+f) - d == f`

浮点数除0的问题

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a=1, b=0;
```

```
    printf( "Division by zero:%d\n ", a/b);
```

```
    getchar();
```

```
    return 0;
```

```
}
```

```
int main()
```

```
{
```

```
    double x=1.0, y=-1.0, z=0.0;
```

```
    printf( "division by zero:%f %f\n ", x/z, y/z);
```

```
    getchar();
```

```
    return 0;
```

```
}
```

这是网上的一个帖子

为什么整数除0会发生异常?

为什么浮点数除0不会出现异常?

浮点运算中，一个有限数除以0，
结果为正无穷大（负无穷大）

问题一：为什么整除int型会产生错误？是什么错误？

二：用double型的时候结果为1. #INF00和-1. #INF00，作何解释???

举例：Ariana火箭爆炸

- 1996年6月4日，Ariana 5火箭初次航行，在发射仅仅37秒钟后，偏离了飞行路线，然后解体爆炸，火箭上载有价值5亿美元的通信卫星。
- 原因是在将一个64位浮点数转换为16位带符号整数时，产生了溢出异常。溢出的值是火箭的水平速率，这比原来的Ariana 4火箭所能达到的速率高出了5倍。在设计Ariana 4火箭软件时，设计者确认水平速率决不会超出一个16位的整数，但在设计Ariana 5时，他们没有重新检查这部分，而是直接使用了原来的设计。
- 在不同数据类型之间转换时，往往隐藏着一些不容易被察觉的错误，这种错误有时会带来重大损失，因此，编程时要非常小心。

举例：爱国者导弹定位错误

- 1991年2月25日，海湾战争中，美国在沙特阿拉伯达摩地区设置的爱国者导弹拦截伊拉克的飞毛腿导弹失败，致使飞毛腿导弹击中了一个美军军营，杀死了美军28名士兵。其原因是由于爱国者导弹系统时钟内的一个软件错误造成的，引起这个软件错误的原因是浮点数的精度问题。
- 爱国者导弹系统中有一内置时钟，用计数器实现，每隔0.1秒计数一次。程序用0.1的一个24位定点二进制小数x来乘以计数值作为以秒为单位的时间
- 这个x的机器数是多少呢？
- 0.1的二进制表示是一个无限循环序列：

0.00011[0011]..., $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\text{B}$ 。

显然，x是0.1的近似表示， $0.1 - x$

$= 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ [1100]\dots - 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\text{B}$

即为：

$= 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100\ [1100]\dots\text{B}$

$= 2^{-20} \times 0.1 \approx 9.54 \times 10^{-8}$

这就是机器值与真值之间的误差！

举例：爱国者导弹定位错误

已知在爱国者导弹准备拦截飞毛腿导弹之前，已经连续工作了100小时，飞毛腿的速度大约为2000米/秒，则由于时钟计算误差而导致的距离误差是多少？

100小时相当于计数了 $100 \times 60 \times 60 \times 10 = 36 \times 10^5$ 次，因而导弹的时钟已经偏差了 $9.54 \times 10^{-8} \times 36 \times 10^5 \approx 0.343$ 秒

因此，距离误差是 $2000 \times 0.343 \text{秒} \approx 687 \text{米}$

小故事：实际上，以色列方面已经发现了这个问题并于1991年2月11日知会了美国陆军及爱国者计划办公室（软件制造商）。以色列方面建议重新启动爱国者系统的电脑作为暂时解决方案，可是美国陆军方面却不知道每次需要间隔多少时间重新启动系统一次。1991年2月16日，制造商向美国陆军提供了更新软件，但这个软件最终却在飞毛腿导弹击中军营后的一天才运抵部队。

举例：爱国者导弹定位错误

- 若x用float型表示，则x的机器数是什么？0.1与x的偏差是多少？系统运行100小时后的时钟偏差是多少？在飞毛腿速度为2000米/秒的情况下，预测的距离偏差为多少？
 - $0.1 = 0.0\ 0011[0011]B = +1.1\ 0011\ 0011\ 0011\ 0011\ 0011\ 00B \times 2^{-4}$ ，故x的机器数为0 011 1101 1 100 1100 1100 1100 1100 1100
 - Float型仅24位有效位数，后面的有效位全被截断，故x与0.1之间的误差为： $|x-0.1| = 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100\ [1100]...B$ 。这个值等于 $2^{-24} \times 0.1 \approx 5.96 \times 10^{-9}$ 。100小时后时钟偏差 $5.96 \times 10^{-9} \times 36 \times 10^5 \approx 0.0215$ 秒。距离偏差 $0.0215 \times 2000 \approx 43$ 米。比爱国者导弹系统精确约16倍。

举例：爱国者导弹定位错误

- 若用32位二进制定点小数 $x=0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ \text{B}$ 表示0.1，则误差比用float表示误差更大还是更小？
- 当 $x=0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ \text{B}$ 时，与0.1之间的误差约为： $|x-0.1|=0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 001100\ [1100]\dots\text{B}$ 。这个值等于 $2^{-30} \times 0.1 \approx 9.31 \times 10^{-11}$ 。100小时后时钟偏差 $9.31 \times 10^{-11} \times 36 \times 10^5 \approx 0.000335$ 秒。预测的距离偏差仅为 $0.000335 \times 2000 \approx 0.67$ 米。

举例：浮点数运算的精度问题

- 从上述结果可以看出：
 - 用32位定点小数表示0.1，比采用float精度高64倍
 - 用float表示在计算速度上更慢，必须先把计数值转换为IEEE 754格式浮点数，然后再对两个IEEE 754格式的数相乘，故采用float比直接将两个二进制数相乘要慢
- Ariana 5火箭和爱国者导弹的例子带来的启示
 - ✓ 程序员应对底层机器级数据的表示和运算有深刻理解
 - ✓ 计算机世界里，经常是“差之毫厘，失之千里”，需要细心再细心，精确再精确
 - ✓ 不能遇到小数就用浮点数表示，有些情况下（如需要将一个整数变量乘以一个确定的小数常量），可先用一个确定的定点整数与整数变量相乘，然后再通过移位运算来确定小数点

Summary

- IEEE Floating Point has clear mathematical properties
- Represents numbers of form $M \times 2^E$
- One can reason about operations independent of implementation
 - As if computed with perfect precision and then rounded
- Not the same as real arithmetic
 - Violates associativity/distributivity
 - Makes life difficult for compilers & serious numerical applications programmers